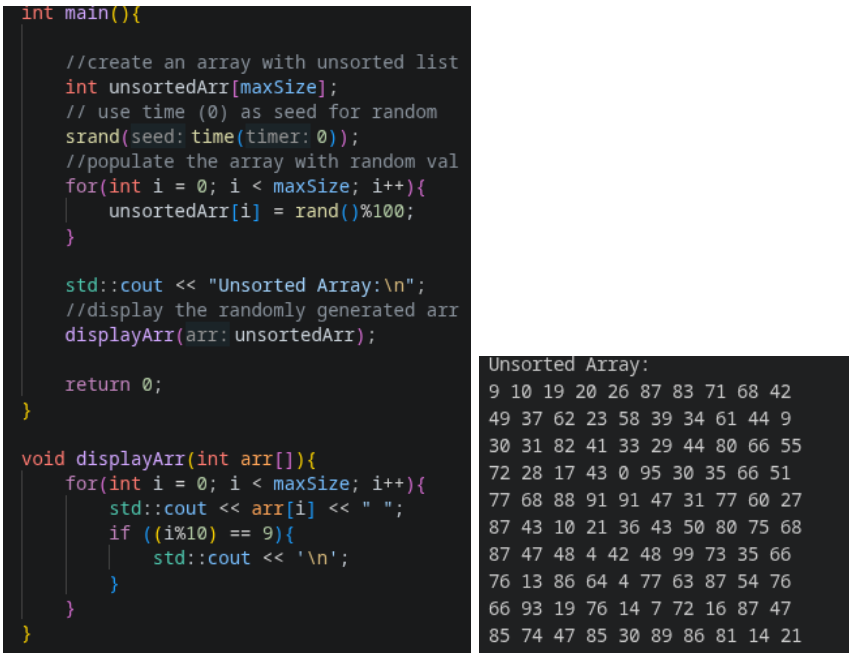
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/9/26	**DATE SUBMITTED:**	9/15/26
NAME:	Condino, Niel Vincent B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 **Sorting Algorithms Pt. 2**

1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	This snippet of the code populates an array with random numbers using the rand() function. This loop runs n times based on the desired size of the array. The displayArr() function also runs O(n) times as it prints out every 10 elements of the array in a row. The combined runtime complexity of the code is O(n) in its worst and best case scenarios. The output is an array populated with pseudorandom numbers generated using the seed of time(0).

```
int main(){

    //create an array with unsorted list
    int unsortedArr[maxSize];
    // use time (0) as seed for random
    srand(seed: time(timer: 0));
    //populate the array with random val
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }

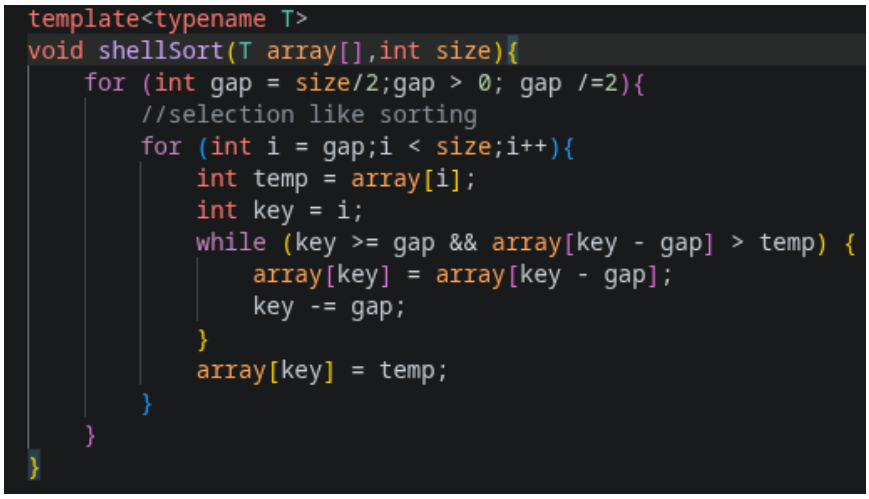
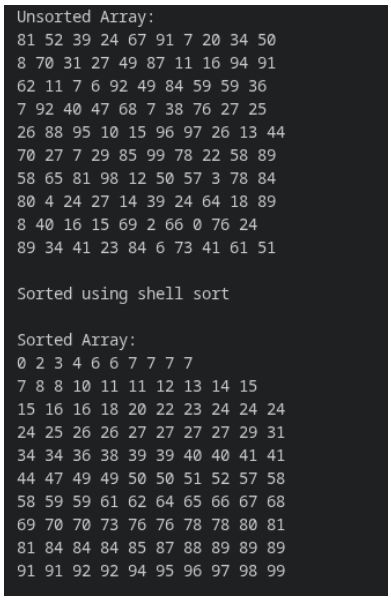
    std::cout << "Unsorted Array:\n";
    //display the randomly generated arr
    displayArr(arr: unsortedArr);

    return 0;
}

void displayArr(int arr[]){
    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
        if ((i%10) == 9){
            std::cout << '\n';
        }
    }
}
```

```
Unsorted Array:
9 10 19 20 26 87 83 71 68 42
49 37 62 23 58 39 34 61 44 9
30 31 82 41 33 29 44 80 66 55
72 28 17 43 0 95 30 35 66 51
77 68 88 91 91 47 31 77 60 27
87 43 10 21 36 43 50 80 75 68
87 47 48 4 42 48 99 73 35 66
76 13 86 64 4 77 63 87 54 76
66 93 19 76 14 7 72 16 87 47
85 74 47 85 30 89 86 81 14 21
```

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	 
Observations	This algorithm is like an insertion sort. The difference between this and the other one is that this algorithm incorporates a gap for its shifting process. This gap gets smaller at each iteration until it becomes zero, and then it turns into an insertion sort. This algorithm has a best-case O(n log n) time complexity when the array is already or nearly sorted. However, this has an $O(n^2)$ worst-case time complexity using the default gap, which is half of the array.

```
template<typename T>
void shellSort(T array[],int size){
    for (int gap = size/2;gap > 0; gap /=2){
        //selection like sorting
        for (int i = gap;i < size;i++){
            int temp = array[i];
            int key = i;
            while (key >= gap && array[key - gap] > temp) {
                array[key] = array[key - gap];
                key -= gap;
            }
            array[key] = temp;
        }
    }
}
```

```
Unsorted Array:
81 52 39 24 67 91 7 20 34 50
8 70 31 27 49 87 11 16 94 91
62 11 7 6 92 49 84 59 59 36
7 92 40 47 68 7 38 76 27 25
26 88 95 10 15 96 97 26 13 44
70 27 7 29 85 99 78 22 58 89
58 65 81 98 12 50 57 3 78 84
80 4 24 27 14 39 24 64 18 89
8 40 16 15 69 2 66 0 76 24
89 34 41 23 84 6 73 41 61 51

Sorted using shell sort

Sorted Array:
0 2 3 4 6 6 7 7 7 7
7 8 8 10 11 11 12 13 14 15
15 16 16 18 20 22 23 24 24 24
24 25 26 26 27 27 27 27 29 31
34 34 36 38 39 39 40 40 41 41
44 47 49 49 50 50 51 52 57 58
58 59 59 61 62 64 65 66 67 68
69 70 70 73 76 76 78 78 80 81
81 84 84 84 85 87 88 89 89 89
91 91 92 92 94 95 96 97 98 99
```

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```cpp
template<typename T>
void merge(T array[],int left,int mid,int right){
    int sizeL = mid-left + 1;
    int sizeR = right - mid;

    T arrL[sizeL];
    T arrR[sizeR];

    for (int i = 0; i < sizeL; i++){
        arrL[i] = array[left + i];
    }

    for (int j = 0; j < sizeR; j++){
        arrR[j] = array[mid + 1 + j];
    }

    int i = 0, j = 0;
    int k = left;

    // Merge the temp arrays back
    // into arr[left..right]
    while (i < sizeL && j < sizeR) {
        if (arrL[i] <= arrR[j]) {
            array[k] = arrL[i];
            i++;
        }
        else {
            array[k] = arrR[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[],
    // if there are any
    while (i < sizeL) {
        array[k] = arrL[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[],
    // if there are any
    while (j < sizeR) {
        array[k] = arrR[j];
        j++;
        k++;
    }

}
```

```cpp
template<typename T>
void mergeSort(T array[],int left,int right){
    if (left >= right)return;
    int mid = left + (right - left) / 2;
    mergeSort(array, left, right: mid);
    mergeSort(array, left: mid + 1, right);
    merge(array, left, mid, right);
}
```

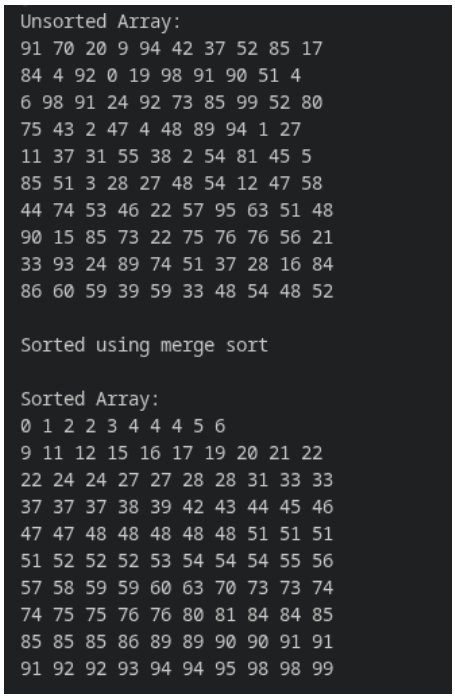
	
Observations	This algorithm tries to halve the array to be sorted by repeatedly dividing the array into two halves. After this is done, the array is then merged in sorted order, giving the lower values a higher priority of being inserted into this new array. It recursively does this until the whole array is rebuilt. This gives an average O(n log n) runtime complexity for sorting arrays. The log n comes from the dividing part, while the n part comes from the merging phase of the algorithm.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot

```cpp
template<typename T>
int partition(T arr[], int low, int high) {

    // choose the pivot
    T pivot = arr[high];

    // undex of smaller element and indicates
    // the right position of pivot found so far
    int i = low - 1;

    // Traverse arr[low..high] and move all smaller
    // elements on left side. Elements from low to
    // i are smaller after every iteration
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    // move pivot after smaller elements and
    // return its position
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

template<typename T>
// the QuickSort function implementation
void quickSort(T arr[], int low, int high) {
    if (low < high) {

        // pi is the partition return index of pivot
        int pi = partition(arr, low, high);

        // recursion calls for smaller elements
        // and greater or equals elements
        quickSort(arr, low, high: pi - 1);
        quickSort(arr, low: pi + 1, high);
    }
}
```

```
Unsorted Array:
5 73 46 46 98 9 47 41 17 8
1 43 63 21 8 20 60 21 54 82
80 32 37 10 92 85 34 57 49 39
60 6 12 58 4 62 68 52 3 37
12 56 32 27 78 93 48 90 14 2
72 94 87 9 56 31 47 42 88 48
81 1 55 45 11 11 8 79 63 63
68 28 20 1 55 50 94 55 40 60
58 12 7 97 73 63 28 20 6 69
69 39 70 76 37 81 87 97 13 3

Sorted using quick sort

Sorted Array:
1 1 1 2 3 3 4 5 6 6
7 8 8 8 9 9 10 11 11 12
12 12 13 14 17 20 20 20 21 21
27 28 28 31 32 32 34 37 37 37
39 39 40 41 42 43 45 46 46 47
47 48 48 49 50 52 54 55 55 55
56 56 57 58 58 60 60 60 62 63
63 63 63 68 68 69 69 70 72 73
73 76 78 79 80 81 81 82 85 87
87 88 90 92 93 94 94 97 97 98
```

Observations	This algorithm used a pivot as a reference. The values around this pivot are moved around based on their comparison to the pivot. The greater items move to the right, while smaller values move to the left. This is done recursively until the sub arrays zero or one element, in which case the array is sorted. This algorithm has a best-case time complexity of O(n log n), which can degrade to $O(n^2)$ with pivots that cause uneven splits.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
template <typename T>
void partiMerge(T arr[],int low,int high){
    int pi = partition(arr, low, high);

    // recursion calls for smaller elements
    // and greater or equals elements
    mergeSort(array: arr, left: low, right: pi - 1);
    mergeSort(array: arr, left: pi + 1, right: high);
}

template<typename T>
void partiShell(T arr[],int size){
    int pi = partition(arr, low: 0, high: size-1);

    shellSort(array: arr, size: pi,start: 0);
    shellSort(array: arr,size,start: pi);
}
```

```
Unsorted Array:
99 54 60 86 84 85 2 27 76 64
80 20 60 40 63 23 63 16 67 98
21 96 59 27 16 91 80 94 46 65
68 72 17 6 95 88 44 92 12 90
12 78 97 89 58 6 47 10 71 1
77 36 77 76 0 71 1 9 77 46
4 66 87 68 98 29 62 49 46 33
92 11 57 48 70 22 51 30 17 0
19 91 47 24 17 44 14 7 79 58
2 79 21 43 7 6 73 11 41 18

Sorted using partition shell and merge

Sorted Array 1:
0 0 1 1 2 2 4 6 6 6
7 7 9 10 11 11 12 12 14 16
16 17 17 17 18 19 20 21 21 22
23 24 27 27 29 30 33 36 40 41
43 44 44 46 46 46 47 47 48 49
51 54 57 58 58 59 60 60 62 63
63 64 65 66 67 68 68 70 71 71
72 73 76 76 77 77 77 78 79 79
80 80 84 85 86 87 88 89 90 91
91 92 92 94 95 96 97 98 98 99

Sorted Array 2:
0 0 1 1 2 2 4 6 6 6
7 7 9 10 11 11 12 12 14 16
16 17 17 17 18 19 20 21 21 22
23 24 27 27 29 30 33 36 40 41
43 44 44 46 46 46 47 47 48 49
51 54 57 58 58 59 60 60 62 63
63 64 65 66 67 68 68 70 71 71
72 73 76 76 77 77 77 78 79 79
80 80 84 85 86 87 88 89 90 91
91 92 92 94 95 96 97 98 98 99
```

Figure 1: Partition using different sorting algorithms

Yes, we can implement a different sorting algorithm when partitioning the array into two halves. Above is an example of this; After partitioning the array, I called the functions for the different sorting algorithms other than the quick sort() function. The runtime of this modified algorithm appears to still have its normal runtimes, both being O(n log n). They both still sorted the array given to them.

```
int main(){

    //create an array with unsorted list
    int unsortedArr[maxSize] ={[0]=4, [1]=34, [2]=29, [3]=48, [4]=53, [5]=87, [6]=12, [7]=30, [8]=44, [9]=25, [10…93, 67, 43, 19,74};
    //populate the array with random val

    std::cout << "Unsorted Array:\n";
    //display the randomly generated arr
    displayArr(arr: unsortedArr);
    std::cout << "\nSorted using quick sort\n";
    quickSort(arr: unsortedArr,low: 0,high: maxSize-1);
    //std::cout << "\nSorted using partition shell and merge\n";

    std::cout << "\nSorted Array :\n";
    displayArr(arr: unsortedArr);

    return 0;
}
```

```
int main(){

    //create an array with unsorted list
    int unsortedArr[maxSize] ={[0]=4, [1]=34, [2]=29, [3]=48, [4]=53, [5]=87, [6]=12, [7]=30, [8]=44, [9]=25, [10…93, 67, 43, 19,74};

    std::cout << "Unsorted Array:\n";
    //display the randomly generated arr
    displayArr(arr: unsortedArr);
    std::cout << "\nSorted using merge sort\n";
    mergeSort(array: unsortedArr,left: 0,right: maxSize-1);
    //std::cout << "\nSorted using partition shell and merge\n";

    std::cout << "\nSorted Array :\n";
    displayArr(arr: unsortedArr);

    return 0;
}
```

```
~/Downloads/HOA/HOA 8
❯ time ./quickTest
Unsorted Array:
4 34 29 48 53 87 12 30 44 25
93 67 43 19 74
Sorted using quick sort

Sorted Array :
4 12 19 25 29 30 34 43 44 48
53 67 74 87 93
________________________________________________________
Executed in    1.45 millis    fish           external
   usr time    0.18 millis  176.00 micros    0.00 millis
   sys time    1.33 millis  115.00 micros    1.22 millis


~/Downloads/HOA/HOA 8
❯ time ./mergeTest
Unsorted Array:
4 34 29 48 53 87 12 30 44 25
93 67 43 19 74
Sorted using merge sort

Sorted Array :
4 12 19 25 29 30 34 43 44 48
53 67 74 87 93
________________________________________________________
Executed in    1.49 millis    fish           external
   usr time    0.29 millis  295.00 micros    0.00 millis
   sys time    1.24 millis    0.00 micros    1.24 millis
```

Figure 2: Quick and Merge Sort Comparison

Between the two, Quick Sort will give the faster practical time performance for this array. While both share an average time complexity of O(n log n), Quick Sort sorts in-place without the extra memory allocation overhead required by Merge Sort. Its smaller constant factors make its operations noticeably faster on data stored in RAM. Both of these algorithms can achieve an O(n log n) runtime because of the divide-and-conquer strategy they apply. The O(log n) runtime comes from them dividing the array into subarrays. The O(n) runtime comes from different operations for each algorithm. It comes from merging in merge sort and partitioning in quick sort.

Conclusion:

In this activity, I was able to learn more sorting algorithms. I consider these new algorithms more advanced than the sorting algorithms in the first part. The algorithms here apply dynamic programming techniques such as recursive calls. Compared to the last activity's algorithms, these algorithms can achieve an O(n log n) runtime, which is faster. I find the implementation of these algorithms more challenging than the first ones, as these algorithms use the said dynamic programming. With the help of visualization from YouTube and other sources, I was able to code these algorithms. The supplementary material also helped me explore the implementations of these algorithms and compare them on a test case. I aim to improve my skills in implementing these kinds of algorithms and when to use them.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.